

Introduction to Sequence to Sequence Models

Overview

Google first released sequence-to-sequence models for machine translation. Before it, the translation operated in an extremely ignorant manner. It was automatically translated into its intended language without considering the syntax or sentence structure of each word you typed. Sequence-to-sequence models used deep learning to transform the translation process. When translating, it considers the current word or input and its surroundings.

It is employed in many applications, including text summarization, conversational modeling, and image captioning.

Introduction to sequence-to-sequence Model

The many-to-many architecture of an RNN is what the sequence-to-sequence model (seq2seq) uses. Its ability to transfer arbitrary-length input sequences to arbitrary-length output sequences has made it useful for various applications. Some of the uses are Language translation, music creation, speech creation, and chatbots are all included in the sequence-to-sequence models concept.

In most cases, the length of the input and output differ. For example, if we take a translation task. Let's assume we need to convert a sentence from English to French.

Consider the sentence "I am doing good" to be mapped to "Je vais bien". We can see that the input feed has four words, but where in the output, we would see only three words. Hence, this algorithm handles these scenarios with varying input and output sequence lengths.

The architecture of the sequence-to-sequence model comprises two components:

- **Encoder**
- **Decoder**

The Encoder learns the embeddings of the input sentence. Embeddings are vectors comprising the meaning of the sentence. Then the Decoder takes the embedding vectors as input and tries constructing the target sentence.

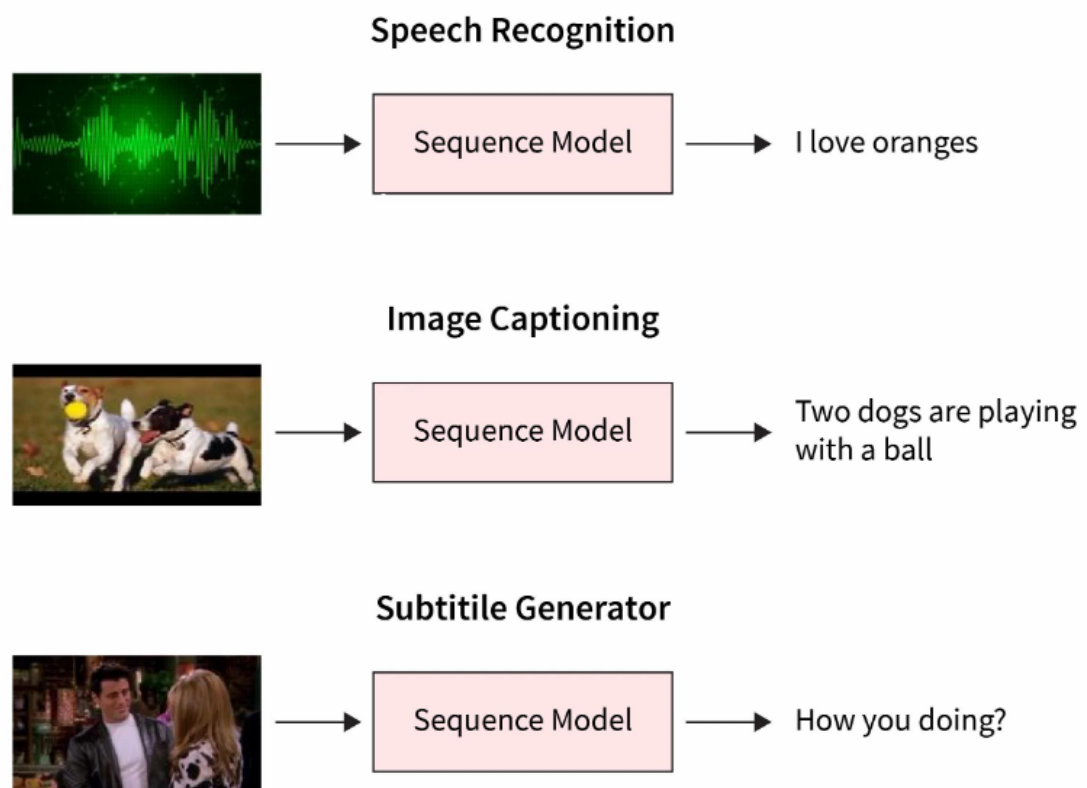
In simple words, in a translation task, the Encoder takes the English sentence as input, learns the embedding from it, and feeds the embeddings to the decoder. The Decoder generates the target French sentence using embedding fed.

Use Cases of the Sequence to Sequence Models

Many of the technologies you use every day are based on sequence-to-sequence models. For instance, voice-activated gadgets, online chatbots, and services like Google Translate are all powered by the sequence-to-sequence model architecture. Among the applications are the following:

- **Machine translation** - The seq2seq model predicts a word from the user's input, after which each subsequent word is predicted using the likelihood that the first word will appear.

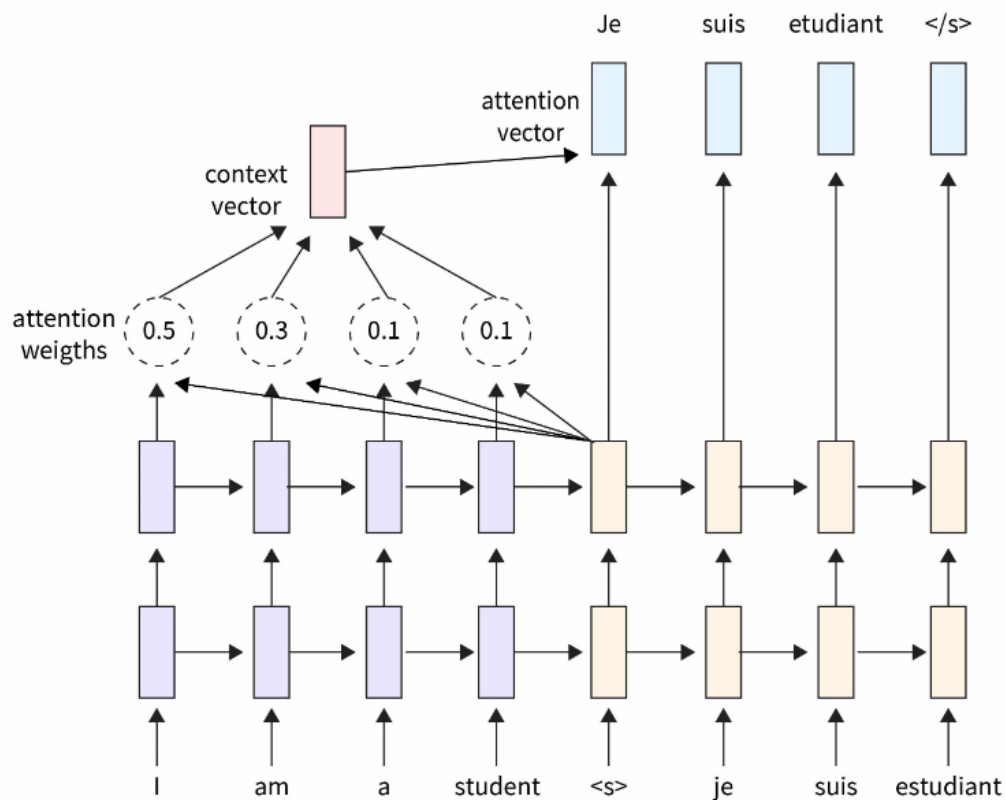
- **Video captioning** - To understand the temporal structure of the sequence of frames and the sequence model of the generated sentences with RNNs, a sequence-to-sequence model for Video captioning was developed.
- **Text summarization** - Using neural sequence-to-sequence models, an effective novel method for abstractive text summarization has been made available (not restricted to selecting and rearranging passages from the original text)



Any sequence-based problem can be solved using this technique, especially if the inputs and outputs come in various sizes and classifications.

Encoder-Decoder Architecture

An overview of the model is illustrated in the diagram below. Both have the Decoder on the right and the Encoder on the left. The Encoder and decoder output are combined to predict the following word at each time step.



One sequence is converted into another via sequence to sequence model (sequence transformation). It uses a recurrent neural network (RNN) or, more frequently, an LSTM or GRU to get over the vanishing gradient issue. The output from the preceding stage serves as the context for each item. One Encoder and one decoder network make up the main parts. First, the Encoder converts each item into a corresponding hidden vector that includes the item and its context. Then, with the previous output as the input context, the Decoder reverses the process and produces the vector as an output item.

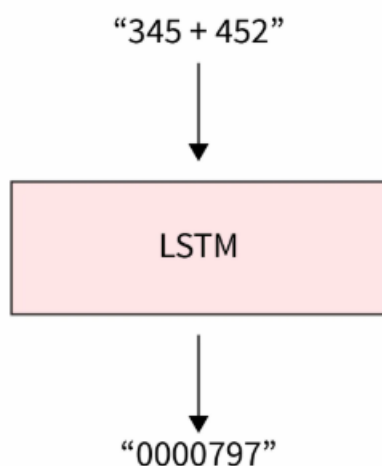
- **Encoder-** Transforms the input words into matching hidden vectors using deep neural network layers. Each vector represents the current word and its context.

- **Decoder-** To create the following hidden vector and ultimately predict the following word, it uses the current word, its hidden states, and the hidden vector created by the Encoder as input. Apart from these two, numerous optimizations must result in the following additional sequence-to-sequence model components.
- **Attention-** The input to the Decoder is a single vector that must contain all of the context-related data. Large sequences cause difficulty with this. As a result, the attention mechanism lets the decoder focus on specific input sequences and generate an attention vector.
- **Beam Search-** The Decoder chooses the word with the highest probability as the output. But because greedy algorithms are the root of the issue, this does not necessarily produce the optimal outcomes. As a result, beam search is used, suggesting potential translations at each stage. A tree of the top k-results is created to do this.
- **Bucketing-** Because both the input and the output of a sequence-to-sequence model are padded with zeroes, variable-length sequences are conceivable. However, if we put the maximum length at 100 and the sentence is just three words long, there will be a significant loss of space. So, we apply the bucketing concept. First, we create buckets of varying sizes, such as (4, 8), (8, 15), and so forth, where 4 is the maximum input length and 8 is the maximum output length.

Input and Output Sequences of Equal Length

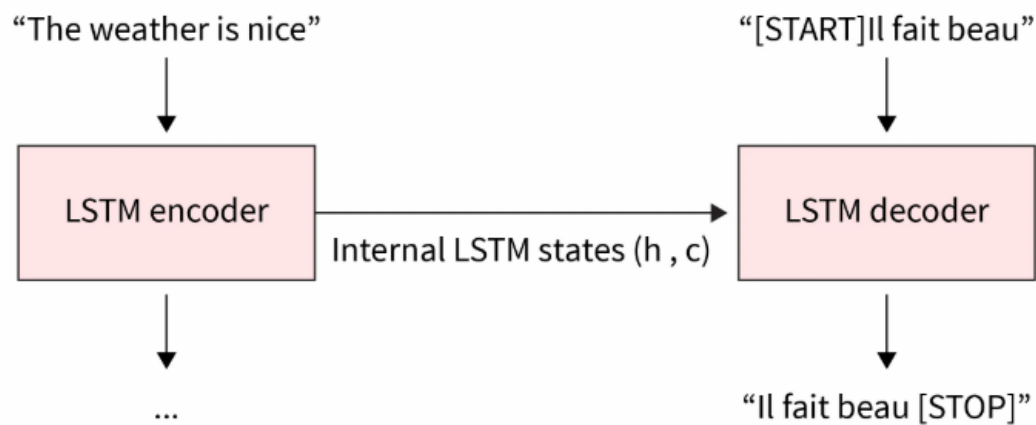
You can easily create similar models with a Keras LSTM or GRU layer when the input and output sequences are the same lengths.

This strategy has a drawback because it presumes you can generate `target[...t]` for given `input[...t]`. Unfortunately, the whole input sequence is usually required before generating the target sequence. That works in a few situations (such as adding strings of digits), but it could be more effective for most use cases.



Language Translation Using the seq2seq Mode

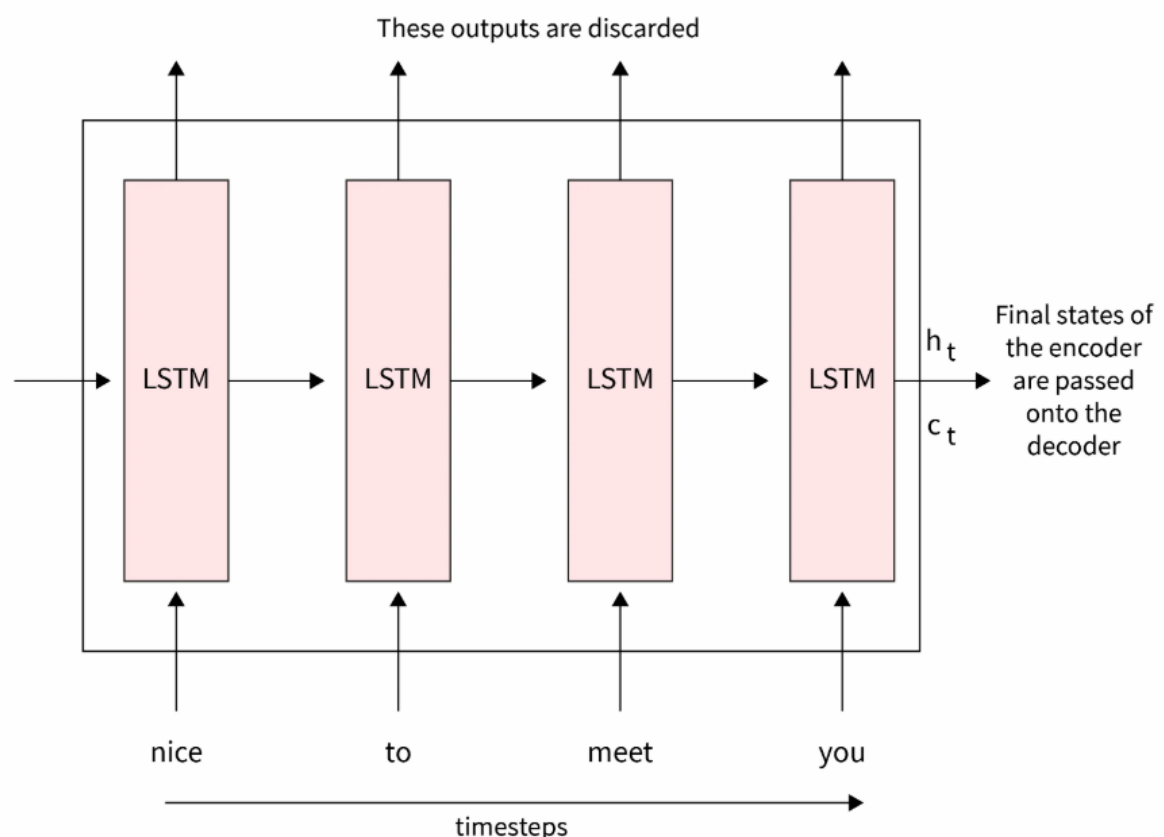
As discussed above, let's learn how the encoder and decoder work to handle tasks that map input and output of different lengths.



We try to translate a sentence from English to French. Let the sentence be, "The weather is nice."

Encoder

The basic structure of an encoder is an RNN with LSTM or GRU cells (control what information to keep or what to throw). Another option is a bidirectional RNN. Instead of using the output from the Encoder, we feed the input sentence into it, and we use the hidden state from the last time step as the embeddings.



As a first step at a time steps $t=0$, To the LSTM cell, we pass the input x_0 , which is the first word from the source sentence. The model randomly initialized the hidden state with the input(h_i); the LSTM cell computes the first hidden state, h_0 , as:

$$h_0 = \text{LSTM}(x_0, h_i)$$

In the next step at time $t=1$, we pass the input, x_1 , the next word from the sentence "weather", to the Encoder. Along with this, we also pass the previously hidden state h_0 to the cell and compute the hidden state h_1 :

$$h_1 = \text{LSTM}(x_1, h_0)$$

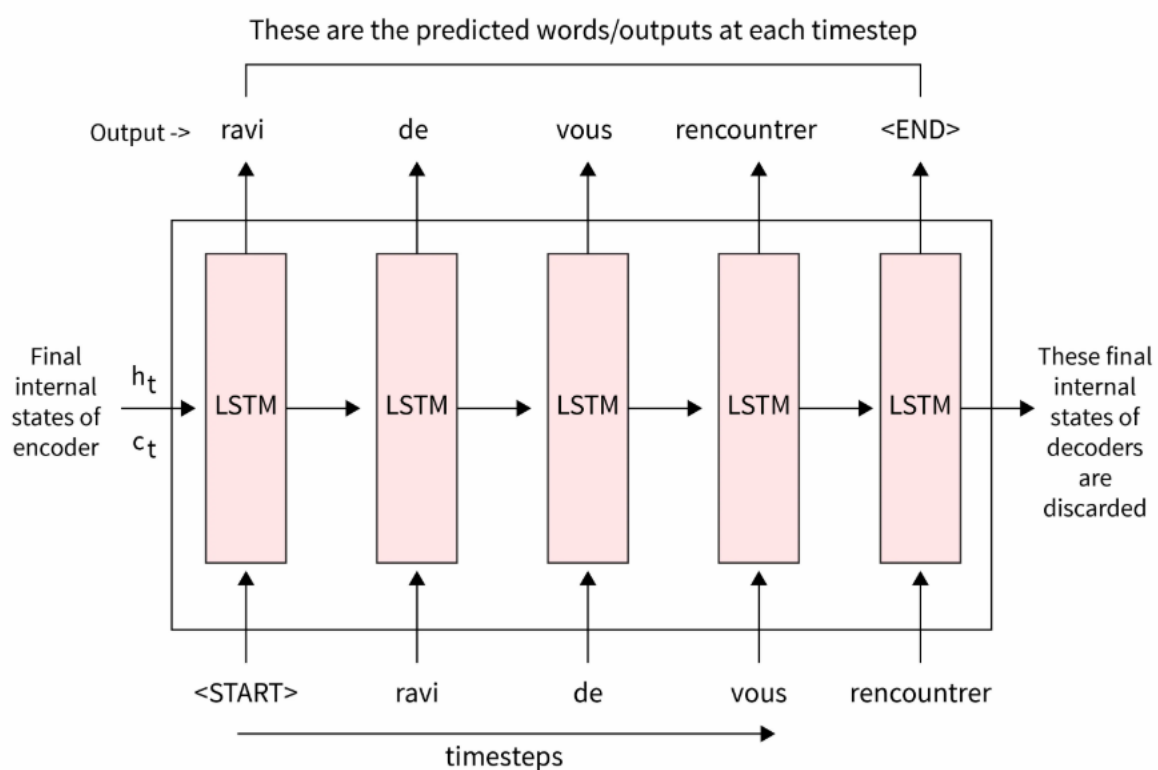
The process repeated all over the source sentence. Thus the final hidden layer captures the context of all the words from the source sentence. Here the English sentence "The weather is nice" is the source. The words processed in the layers would be "The,", "Weather", "is", and "nice".

Since the final hidden layers hold context, this forms the embedding C_t , which is otherwise known as the context vector:

$$C_t = h_t$$

Decoder

A decoder is an RNN with LSTM OR GRU cells. The Decoder aims to generate the output from the given input sentence. Now, we will learn about the working of the Decoder to generate the output using the vector Z generated by the Encoder.



We provide the Decoder with < sos >, which denotes the beginning of the sentence, as input. So, after receiving < sos >, the decoder attempts to guess the target sentence's first word.

We initialize the first hidden state of the Decoder with the vector generated by the Encoder instead of initializing them with a random value. Next, we feed the predicted output and the hidden state from the previous time step as input to the Decoder at the current time step and then predict the current output.

We need to predict the output sentence, a French sentence mapping the English sentence that feeds as input to the model. Then, we feed the Decoder hidden state, which returns the scores of all the words in our vocabulary to the corresponding input.

We convert them into probabilities and use the softmax function, which squashes the values between 0 to 1. Then, we select the high-probability valued word for the mapping to the input.

Thus, the sequence-to-sequence model changes the source sentence into the destination sentence in this manner.

Drawbacks of Encoder-Decoder Models

This architecture has two main disadvantages, both of which are length-related.

- First off, its architecture has considerably less memory. You're attempting to squeeze the full sentence to translate into that last hidden state of the LSTM. Typically, they only have a few hundred units (or floating-point integers); the more you try to fit into this restricted dimensions vector, the lossier the neural network is compelled to be. It can be quite helpful to think of neural networks regarding the "lossy compression" that must be done.
- Second, a neural network is more challenging to train the deeper it goes. For recurrent neural networks, the sequence length determines how far the network extends over time. As a result, the gradient signal from the recurrent neural network learns vanishes as it moves backward, resulting in vanishing gradients. Hence this remains a basic issue even with RNNs designed to help prevent disappearing gradients, like the LSTM.

Conclusion

- Sequence-to-sequence (Seq2Seq) models excelled at tasks including text summarization, image captioning, and machine translation.
- They make handling tasks with different input lengths and the expected output much easier.
- Additionally, models like Attention Models and Transformers are available for longer, more complex sentences.

Understanding Bidirectional RNN

Overview

Bi-directional recurrent neural networks (Bi-RNNs) are artificial neural networks that process input data in both the forward and backward directions. They are often used in natural language processing tasks, such as language translation, text classification, and named entity recognition. In addition, they can capture contextual dependencies in the input data by considering past and future contexts. Bi-RNNs consist of two separate RNNs that process the input data in opposite directions, and the outputs of these RNNs are combined to produce the final output.

What is Bidirectional RNN?

A bi-directional recurrent neural network (Bi-RNN) is a type of recurrent neural network (RNN) that processes input data in both forward and backward directions. The goal of a Bi-RNN is to capture the contextual dependencies in the input data by processing it in both directions, which can be useful in various natural language processing (NLP) tasks.

In a Bi-RNN, the input data is passed through two separate RNNs: one processes the data in the forward direction, while the other processes it in the reverse direction. The outputs of these two RNNs are then combined in some way to produce the final output.

One common way to combine the outputs of the forward and reverse RNNs is to concatenate them. Still, other methods, such as element-wise addition or multiplication, can also be used. The choice of combination method can depend on the specific task and the desired properties of the final output.

Need for Bi-directional RNNs

- A uni-directional recurrent neural network (RNN) processes input sequences in a single direction, either from left to right or right to left.
- This means the network can only use information from earlier time steps when making predictions at later time steps.
- This can be limiting, as the network may not capture important contextual information relevant to the output prediction.
- For example, in natural language processing tasks, a uni-directional RNN may not accurately predict the next word in a sentence if the previous words provide important context for the current word.

Consider an example where we could use the recurrent network to predict the masked word in a sentence.

1. Apple is my favorite ____.
2. Apple is my favourite ____, and I work there.
3. Apple is my favorite ____, and I am going to buy one.

In the first sentence, the answer could be fruit, company, or phone. But it can not be a fruit in the second and third sentences.

A Recurrent Neural Network that can only process the inputs from left to right may not accurately predict the right answer for sentences discussed above.

To perform well on natural language tasks, the model must be able to process the sequence in both directions.

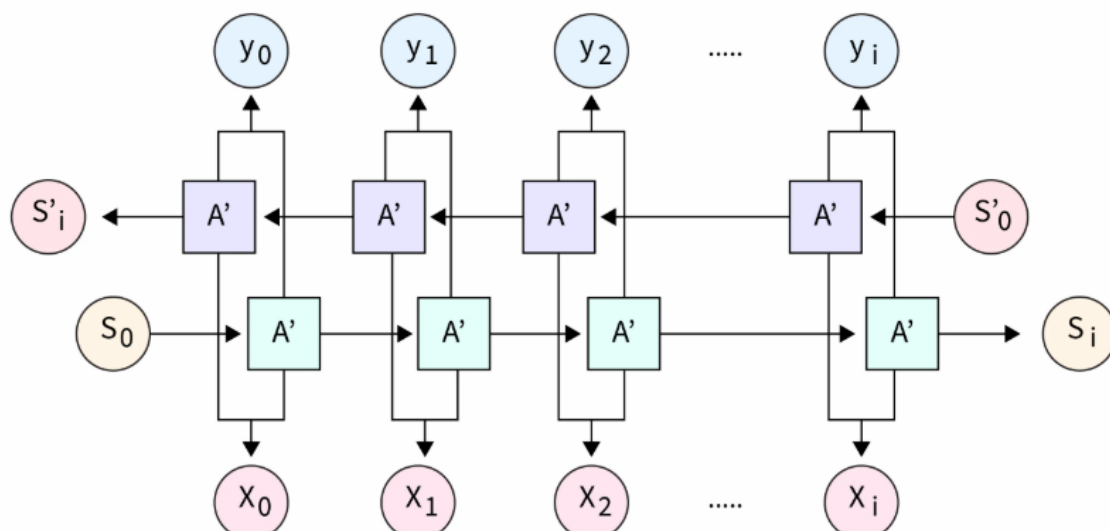
Bi-directional RNNs

- A bidirectional recurrent neural network (RNN) is a type of recurrent neural network (RNN) that processes input sequences in both forward and backward directions.
- This allows the RNN to capture information from the input sequence that may be relevant to the output prediction. Still, the same could be lost in a traditional RNN that only processes the input sequence in one direction.
- This allows the network to consider information from the past and future when making predictions rather than just relying on the input data at the current time step.
- This can be useful for tasks such as language processing, where understanding the context of a word or phrase can be important for making accurate predictions.
- In general, bidirectional RNNs can help improve a model's performance on various sequence-based tasks.

This means that the network has **two separate RNNs**:

1. One that processes the input sequence from left to right
2. Another one that processes the input sequence from right to left.

These two RNNs are typically called forward and backward RNNs, respectively.



During the forward pass of the RNN, the forward RNN processes the input sequence in the usual way by taking the input at each time step and using it to update the hidden state. The updated hidden state is then used to predict the output.

Backpropagation through time (BPTT) is a widely used algorithm for training recurrent neural networks (RNNs). It is a variant of the backpropagation algorithm specifically designed to handle the temporal nature of RNNs, where the output at each time step depends on the inputs and outputs at previous time steps.

In the case of a bidirectional RNN, BPTT involves two separate Backpropagation passes: one for the forward RNN and one for the backward RNN. During the forward pass, the forward RNN processes the input sequence in the usual way and makes predictions for the output sequence. These predictions are then compared to the target output sequence, and the error is backpropagated through the network to update the weights of the forward RNN.

The backward RNN processes the input sequence in reverse order during the backward pass and predicts the output sequence. These predictions are then compared to the target output sequence in reverse order, and the error is backpropagated through the network to update the weights of the backward RNN.

Once both passes are complete, the weights of the forward and backward RNNs are updated based on the errors computed during the forward and backward passes, respectively. This process is repeated for multiple iterations until the model converges and the predictions of the bidirectional RNN are accurate.

This allows the bidirectional RNN to consider information from past and future time steps when making predictions, which can significantly improve the model's accuracy.

Merge Modes in Bidirectional RNN

In a bidirectional recurrent neural network (RNN), two separate RNNs process the input data in opposite directions (forward and backward). The output of these two RNNs is then combined, or "merged," in some way to produce the final output of the model.

There are several ways in which the outputs of the forward and backward RNNs can be merged, depending on the specific needs of the model and the task it is being used for. Some common merge modes include:

1. **Concatenation:** In this mode, the outputs of the forward and backward RNNs are concatenated together, resulting in a single output tensor that is twice as long as the original input.
2. **Sum:** In this mode, the outputs of the forward and backward RNNs are added together element-wise, resulting in a single output tensor that has the same shape as the original input.
3. **Average:** In this mode, the outputs of the forward and backward RNNs are averaged element-wise, resulting in a single output tensor that has the same shape as the original input.

4. **Maximum:** In this mode, the maximum value of the forward and backward outputs is taken at each time step, resulting in a single output tensor with the same shape as the original input.

Which merge mode to use will depend on the specific needs of the model and the task it is being used for. Concatenation is generally a good default choice and works well in many cases, but other merge modes may be more appropriate for certain tasks.

What's the Difference Between BRNN and Recurrent Neural Network?

Unlike standard recurrent neural networks, BRNN's are trained to simultaneously predict both the positive and negative directions of time.

Characteristic	BRNN	RNN
Definition	Bidirectional Recurrent Neural Networks.	Recurrent Neural Networks.
Purpose	Process input sequences in both forward and backward directions.	Process input sequences in a single direction.
Output	Output at each time step depends on the past and future inputs.	Output at each time step depends only on the past inputs.
Training	Trained on both forward and backward sequences.	Trained on a single sequence.
Examples	Natural language processing tasks, speech recognition.	Time series prediction, language translation.

Deep Recurrent Neural Network

<https://www.youtube.com/watch?v=mIDkTrlLaio>

Generating Images Using GANs in TensorFlow

Overview

This article explains using a **Generative Adversarial Network (GAN)** to generate new images of handwritten digits. A GAN is a machine-learning model consisting of a **Generator** and a **Discriminator**.

The Generator creates novel images randomly, while the Discriminator attempts to prove that the generated images are fake. The final output of the GAN is a batch of images that look like handwritten digits. This article explains creating and testing a GAN on **MNIST** data.

What are We Building?

Creating novel images given an image dataset is one of the strengths of a specific branch of models called Generative Adversarial Networks (GAN). These networks specialize in unsupervised/semi-supervised image generation given any image data.

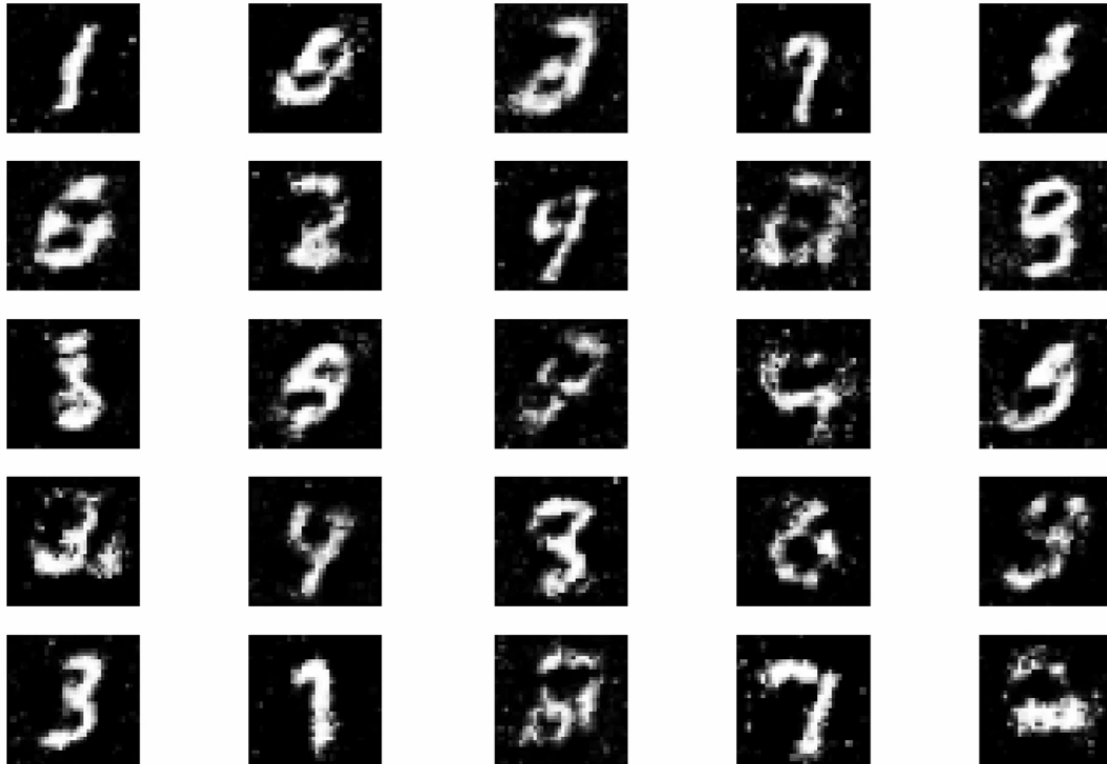
This article uses the GANs image generation ability to create novel handwritten digits. We perform this generation by training the network on a dataset of handwritten digits. We will create a simple GAN from scratch using the Tensorflow library, train it on the MNIST dataset and generate new images of handwritten digits.

How are We Going to Build This?

In this article, we focus on the GAN's image generation ability. We must first load and preprocess an image dataset to let the GAN learn about images. After loading the data, we must create the GAN and write the training and testing code. The below sections focus on implementing these features and generating new images from the MNIST dataset.

Final Output

Our final output should be a batch of images that look like handwritten digits. After training the GAN for 10000 epochs on the MNIST dataset, the image below is what we get.



Requirements

Before creating the GAN's image generation module, we must import a few libraries.

- We will import all the functions, layers, and dataset loaders from TensorFlow.
- We will also import numpy (a math library) and matplotlib (a plotting library).

Building the Model

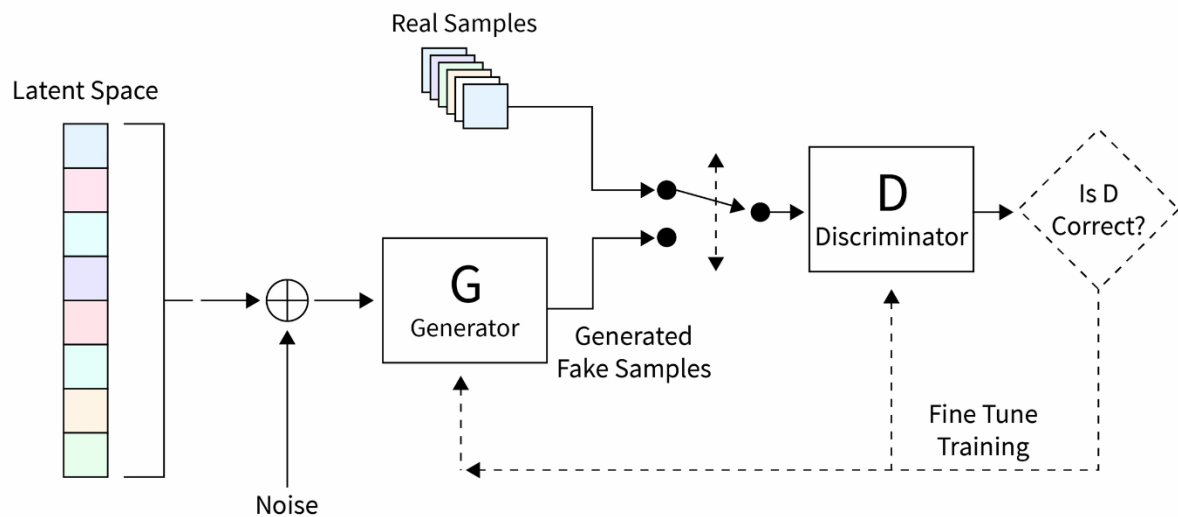
The GAN we want to create comprises two major parts:

- Generator
- Discriminator.

The **Generator** is responsible for creating novel images, while the **Discriminator** is responsible for understanding how good the generated image is.

The entire architecture we want to build for the GANs image generation is shown in the following diagram.

Generative Adversarial Network



The sections below explain how to read a dataset, create the required architecture, compute the loss functions, and train the network. Finally, the code to test the network and create new images is also shown.

Reading the Dataset

This article will use the **MNIST** (Modified National Institute of Standards and Technology) dataset. This dataset has more handwritten digits of 28x28 and is one of computer vision's most widely used datasets. The MNIST is an easy dataset for a GAN such as the one we are building, as it has small, single channel images.

A sample of the dataset is shown below.

We only need to write a little code to load the MNIST dataset as Tensorflow comes with it inbuilt. After loading the dataset, we normalize and reshape it into three dimensions. This reshaping enables the GAN architecture to use this 2D data. We also pre-allocate some memory for our training and validation data.

The shape of the image is defined as a matrix of 28x28x1. The last dimension corresponds to the number of channels in an image. Since we are using the MNIST dataset in black and white, we only have a single channel.

The **zsize** is the shape of the latent space we want to generate. In this case, we set it to 100. This number could be modified if required.

```
from __future__ import print_function, division
```

```
from keras.datasets import mnist
```

```
from keras.layers import Input, Dense, Reshape, Flatten, Dropout

from keras.layers import BatchNormalization, Activation, ZeroPadding2D

from keras.layers import LeakyReLU

from keras.layers.convolutional import UpSampling2D, Conv2D

from keras.models import Sequential, Model

from keras.optimizers import Adam, SGD
```

```
import matplotlib.pyplot as plt

import sys

import numpy as np
```

```
num_rows = 28

num_cols = 28

num_channels = 1

input_shape = (num_rows, num_cols, num_channels)

z_size = 100
```

```
(train_ims, _), (_, _) = mnist.load_data()

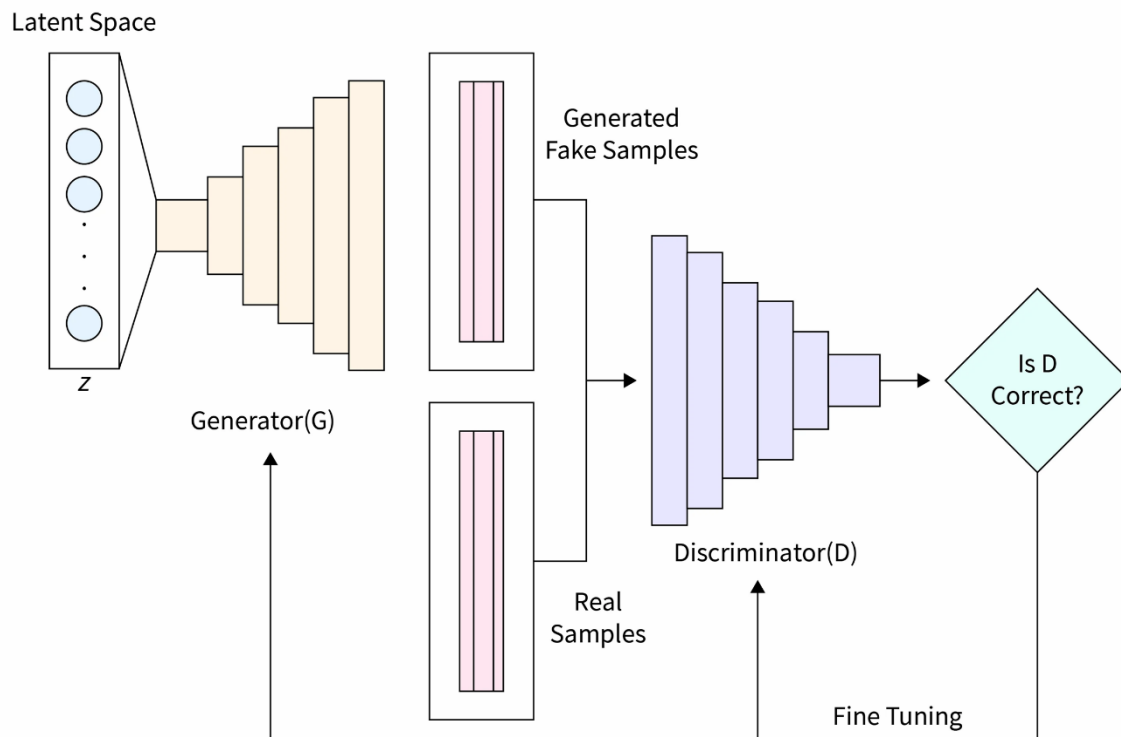
train_ims = train_ims / 127.5 - 1.

train_ims = np.expand_dims(train_ims, axis=3)
```

```
valid = np.ones((batch_size, 1))

fake = np.zeros((batch_size, 1))
```

Defining the Generator



The job of the Generator (D) is to create realistic images that the Discriminator fails to understand are fake. Thus, the Generator is an essential component that enables a GANs image generation ability. The architecture we consider in this article comprises fully connected layers (FC) and Leaky ReLU activations. The final layer of the Generator has a TanH activation rather than a LeakyReLU. This replacement was done because we wanted to convert the generated image to the same range as the original MNIST dataset (-1, 1).

```
def build_generator():

    gen_model = Sequential()

    gen_model.add(Dense(256, input_dim=z_size))

    gen_model.add(LeakyReLU(alpha=0.2))

    gen_model.add(BatchNormalization(momentum=0.8))

    gen_model.add(Dense(512))

    gen_model.add(LeakyReLU(alpha=0.2))

    gen_model.add(BatchNormalization(momentum=0.8))

    gen_model.add(Dense(1024))

    gen_model.add(LeakyReLU(alpha=0.2))
```



```

gen_model.add(BatchNormalization(momentum=0.8))

gen_model.add(Dense(np.prod(input_shape), activation='tanh'))

gen_model.add(Reshape(input_shape))


gen_noise = Input(shape=(z_size,))

gen_img = gen_model(gen_noise)

return Model(gen_noise, gen_img)

```

Defining the Discriminator

The GAN uses the Discriminator (D) to identify how real the Generator's outputs look by returning a probability of real vs. fake. This part of the network can be considered a binary classification problem. To solve this binary classification problem, we need a rather simple network composed of blocks of Fully Connect Layers (FC), Leaky ReLU activations, and Dropout layers. Note that the final layer has a block with an FC layer and a Sigmoid. The final Sigmoid activation returns the classification probability that we require.

```

def build_discriminator():

    disc_model = Sequential()

    disc_model.add(Flatten(input_shape=input_shape))

    disc_model.add(Dense(512))

    disc_model.add(LeakyReLU(alpha=0.2))

    disc_model.add(Dense(256))

    disc_model.add(LeakyReLU(alpha=0.2))

    disc_model.add(Dense(1, activation='sigmoid'))


    disc_img = Input(shape=input_shape)

    validity = disc_model(disc_img)

    return Model(disc_img, validity)

```

Computing the Loss Function

To make the GANs image generation procedure smoother, we need to supply it with metrics showing how well it performs now. Loss functions do just that.

The Discriminator classifies the generated images into real or fake and returns the probability of it being real. To make this distinction, it must ensure that the input it receives is part of the real dataset. And if the input received is fake, it is not classified as part of the real dataset. We can mathematically understand this difference as **maximizing** $D(x)D(x)$ and **minimizing** $D(G(z))D(G(z))$.

Building on these concepts, the Generator is tasked with fooling the Discriminator by creating realistic images. We can understand this procedure as ensuring that when the Discriminator gets an image sampled from the fake dataset, it thinks that the image belongs to the real dataset instead. We can mathematically understand this procedure as **maximizing** $D(G(z))$. Using this part of the formulae as a loss function sometimes makes the network confident about the wrong outputs. To prevent this assumption, we use $\log(D(G(z)))$ instead.

The net cost function for the GAN's image generation can be thus mathematically represented as $\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log(D(x))] + E_{z \sim p_z} [\log(1 - D(G(z)))]$

Training a GAN such as this is a delicate balance and can be considered a game between two enemies. (Hence the name - **Adversarial Learning**.) Since either party attempts to influence the opposition and reduce the others' chance of winning, this is a **MinMax game**.

We can create the Generator and Discriminator with a **Binary Crossentropy loss**.

```
# discriminator
```

```
disc= build_discriminator()
```

```
disc.compile(loss='binary_crossentropy',
```

```
    optimizer='sgd',
```

```
    metrics=['accuracy'])
```

```
z = Input(shape=(z_size,))
```

```
# generator
```

```
img = generator(z)
```

```
disc.trainable = False
```

```
validity = disc(img)
```

```
# combined model
```

```
combined = Model(z, validity)
```

```
combined.compile(loss='binary_crossentropy', optimizer='sgd')
```

Optimizing the Loss

To train the network, we need the GAN to play the **MinMax game**. The training procedure hinges on performing Gradient Descent on the network weights. To reduce the training time and ensure that the training does not get stuck on the loss landscape, we use a Stochastic version of GD, aka **Stochastic Gradient Descent**.

Both the Discriminator and the Generator have **different losses**. If We gave both these networks a single loss function, they would not be able to optimize each other.

```
def initialize_model():
```

```
    disc= build_discriminator()
```

```
    disc.compile(loss='binary_crossentropy',
```

```
                optimizer='sgd',
```

```
                metrics=['accuracy'])
```

```
    generator = build_generator()
```

```
    z = Input(shape=(z_size,))
```

```
    img = generator(z)
```

```
    disc.trainable = False
```

```
    validity = disc(img)
```

```
    combined = Model(z, validity)
```

```
    combined.compile(loss='binary_crossentropy', optimizer='sgd')
```

```
    return disc, Generator, and combined
```

Having defined all the required functions, we can train the network to optimize the losses. The steps we follow for the GAN's image generation are as follows.

- Load an image, and generate random noise the same size as the loaded image.
- Send these images to the Discriminator and calculate the real vs. fake probability.
- Generate another noise of the same size. Send this noise to the Generator.
- Run training for the Generator for a few epochs.
- Repeat all the steps until a satisfactory image is generated.

These steps are directly translated into the code shown below.

```
def train(epochs, batch_size=128, sample_interval=50):

    # load images

    (train_ims, _), (_, _) = mnist.load_data()

    # preprocess

    train_ims = train_ims / 127.5 - 1.

    train_ims = np.expand_dims(train_ims, axis=3)


    valid = np.ones((batch_size, 1))

    fake = np.zeros((batch_size, 1))

    # training loop

    for epoch in range(epochs):

        batch_index = np.random.randint(0, train_ims.shape[0], batch_size)

        imgs = train_ims[batch_index]

        # create noise

        noise = np.random.normal(0, 1, (batch_size, z_size))

        # predict using a Generator

        gen_imgs = gen.predict(noise)

        # calculate loss functions

        real_disc_loss = disc.train_on_batch(imgs, valid)

        fake_disc_loss = disc.train_on_batch(gen_imgs, fake)
```

```

disc_loss_total = 0.5 * np.add(real_disc_loss, fake_disc_loss)

noise = np.random.normal(0, 1, (batch_size, z_size))

g_loss = full_model.train_on_batch(noise, valid)

# show progress

print ("%d [D loss: %f, acc.: %.2f%%] [G loss: %f]" % (epoch, disc_loss_total[0],
100*disc_loss_total[1], g_loss))

# save outputs every few epochs

if epoch % sample_interval == 0:

    one_batch(epoch)

```

Generating Handwritten Digits

Finally, we can generate handwritten digits from the MNIST dataset. To look at how far the network has trained the images, we create a helper function to store predictions from the Generator for a batch of images. This function creates random noise, passes them to the Generator, processes it for display, and then saves it to a folder. We run this helper function every 200 epochs.

```

def one_batch(epoch):

    r, c = 5, 5

    noise = np.random.normal(0, 1, (r * c, z_size))

    gen_imgs = gen.predict(noise)

    # Rescale images 0 - 1

    gen_imgs = 0.5 * gen_imgs + 0.5

    fig, axs = plt.subplots(r, c)

    cnt = 0

    for i in range(r):

        for j in range(c):

            axs[i,j].imshow(gen_imgs[cnt, :,0], cmap='gray')

```

```

    axs[i,j].axis('off')

    cnt += 1

fig.savefig("images/%d.png" % epoch)

plt.close()

```

For this article, we trained the GAN for around 10,000 epochs with a batch size of 32. We save the generated images every 200 epochs in the images folder.

```

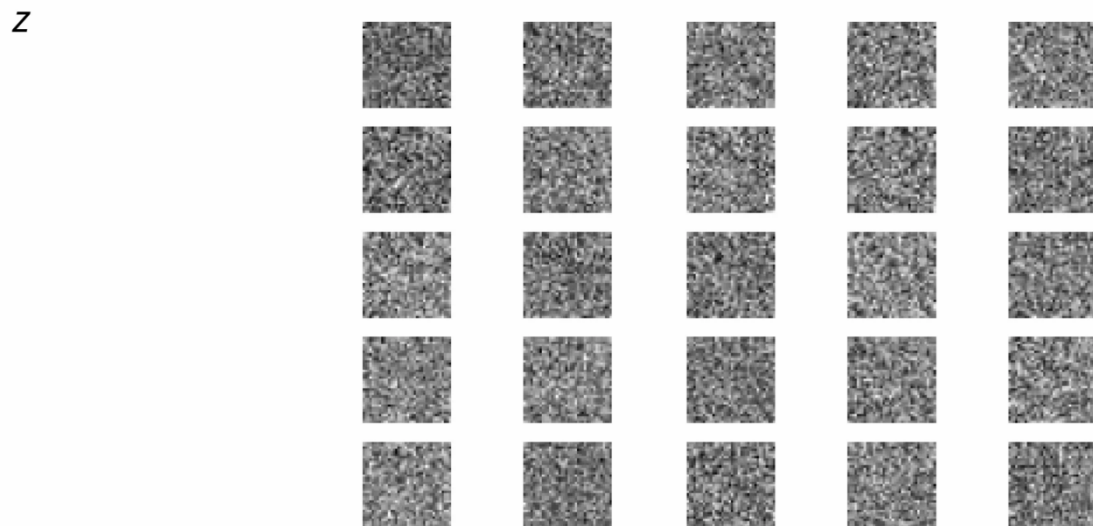
disc, gen, full_model = initialize_model()

train(epochs=10000, batch_size=32, sample_interval=200)

```

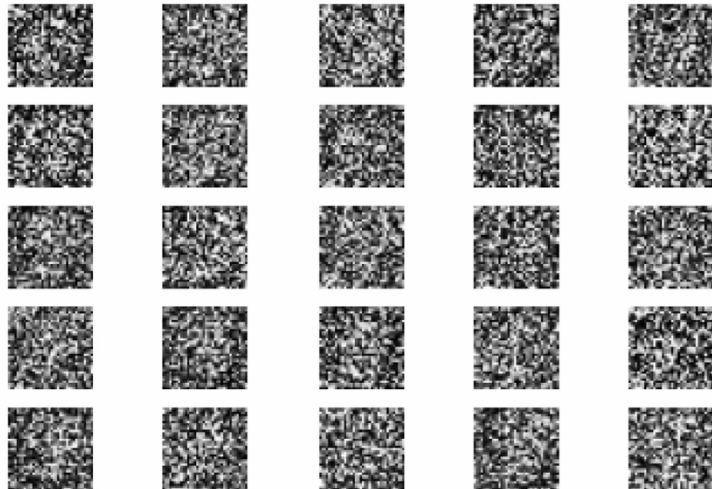
We can now look at the results of the GAN's image generation at the start, at 400 epochs, at 5000 epochs, and the final result at 10000 epochs.

At the start, we have random noise.



After the 400 epochs, we are getting somewhere slowly. But these results are different from real digits.

Z



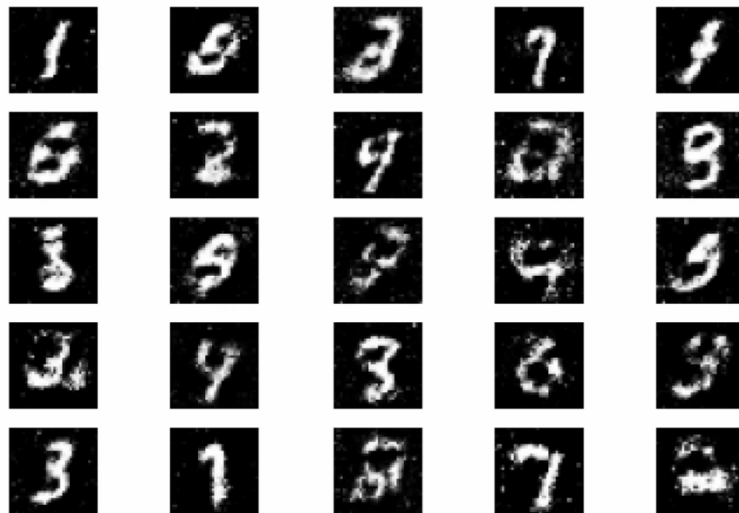
After the 5000 epochs, we can see figures that resemble the MNIST dataset.

Z



After training the network for the entire 10,000 epochs, we get the following outputs.

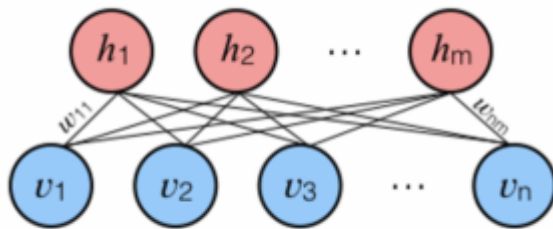
z



These images look very close to the handwritten number data we fed the network. These images were not shown to the network during training and were generated from scratch.

Restricted Boltzmann Machines:

What are Restricted Boltzmann Machines?



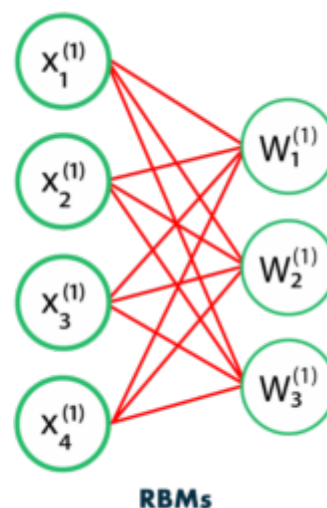
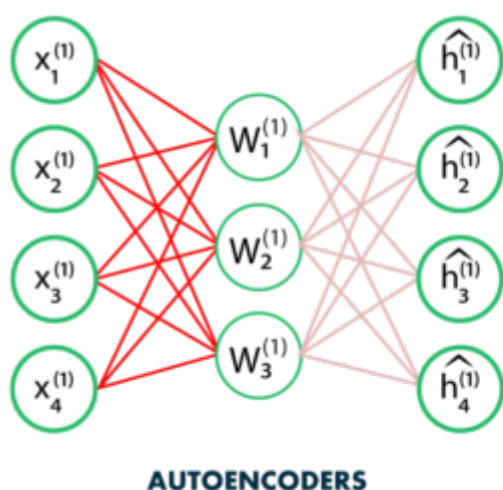
Restricted Boltzmann Machine is an undirected graphical model that plays a major role in Deep Learning Framework in recent times. It was initially introduced as **Harmonium** by Paul Smolensky in 1986 and it gained big popularity in recent years in the context of the [Netflix Prize](#) where Restricted Boltzmann Machines achieved state of the art performance in collaborative filtering and have beaten most of the competition.

It is an algorithm which is useful for dimensionality reduction, classification, regression, collaborative filtering, feature learning, and topic modeling.

Now let's see how Restricted Boltzmann machine differs from other Autoencoder.

Difference between Autoencoders & RBMs

[Autoencoder](#) is a simple 3-layer neural network where output units are directly connected back to input units. Typically, the number of hidden units is much less than the number of visible ones. The task of training is to minimize an error or reconstruction, i.e. find the most efficient compact representation for input data.



RBM shares a similar idea, but it uses stochastic units with particular distribution instead of deterministic distribution. The task of training is to find out how these two sets of variables are actually connected to each other.

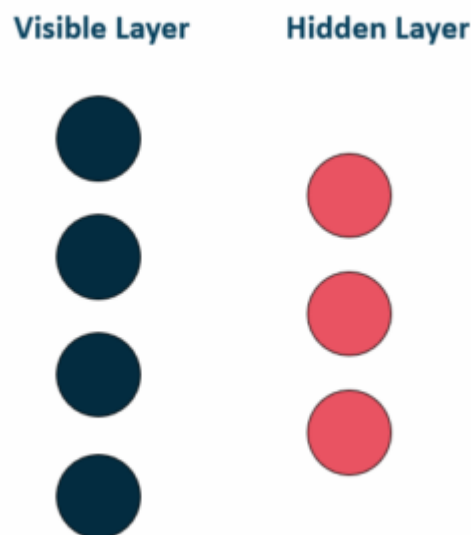
One aspect that distinguishes RBM from other autoencoders is that it has **two biases**.

- The hidden bias helps the RBM produce the activations on the **forward pass**, while
- The visible layer's biases help the RBM learn the reconstructions on the **backward pass**.

Now that we know what is Restricted Boltzmann Machine and what are the differences between RBM and Autoencoders, let's continue with our Restricted Boltzmann Machine Tutorial and have a look at their architecture and working.

Layers in Restricted Boltzmann Machine

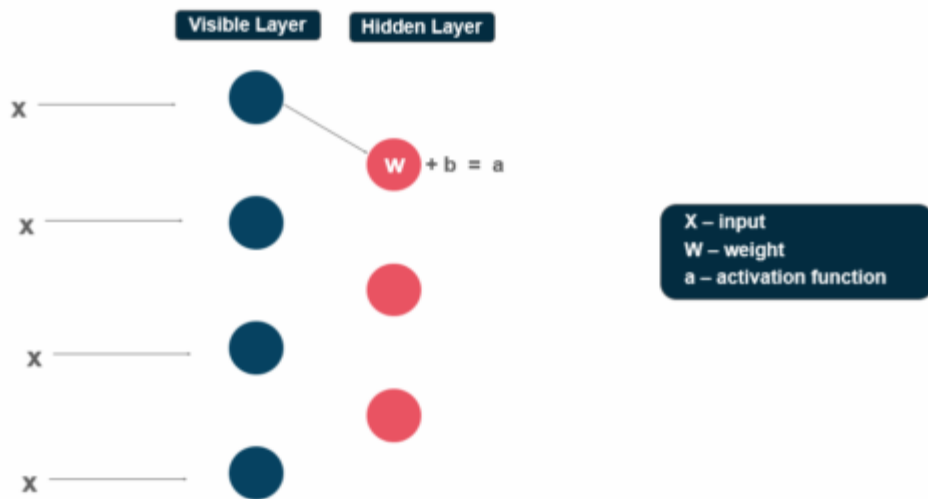
Restricted Boltzmann Machines are shallow, two-layer neural nets that constitute the building blocks of **deep-belief networks**. The first layer of the RBM is called the **visible**, or input layer, and the second is the **hidden** layer. Each circle represents a neuron-like unit called a node. The nodes are connected to each other across layers, but no two nodes of the same layer are linked.



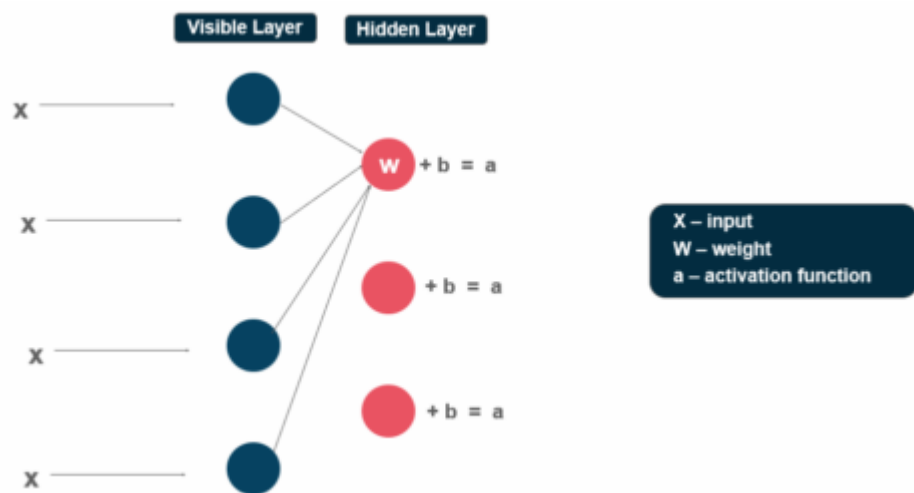
The **restriction** in a Restricted Boltzmann Machine is that there is **no intra-layer communication**. Each node is a locus of computation that processes input and begins by making stochastic decisions about whether to transmit that input or not.

Working of Restricted Boltzmann Machine

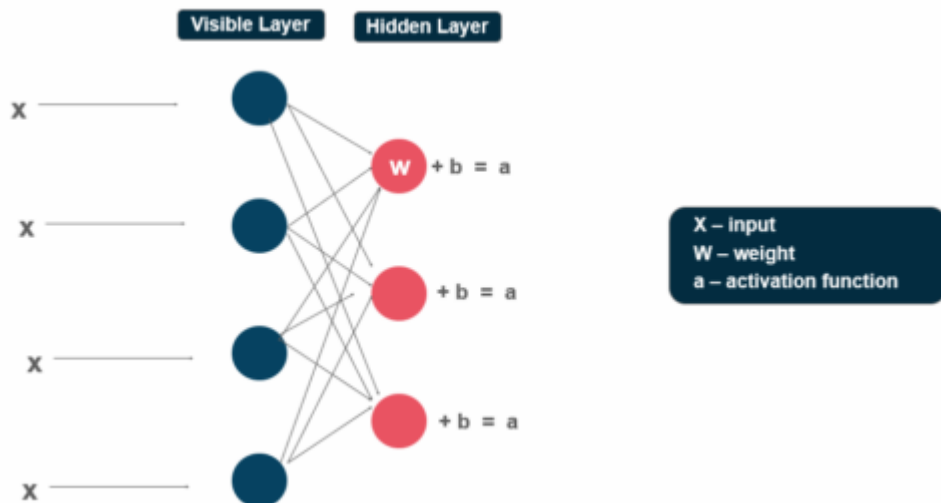
Each visible node takes a low-level feature from an item in the dataset to be learned. At node 1 of the hidden layer, **x** is **multiplied** by a **weight** and added to a **bias**. The result of those two operations is fed into an **activation function**, which produces the node's output, or the strength of the signal passing through it, given input x.



Next, let's look at how several inputs would combine at one hidden node. Each **x** is **multiplied** by a separate weight, the products are summed, added to a bias, and again the result is passed through an activation function to produce the node's output.



At each hidden node, each input **x** is **multiplied** by its respective weight w . That is, a single input x would have three weights here, making 12 weights altogether (4 input nodes x 3 hidden nodes). The weights between the two layers will always form a matrix where the rows are equal to the input nodes, and the columns are equal to the output nodes.



Each hidden node receives the four inputs multiplied by their respective weights. **The sum** of those products is again **added to a bias** (which forces at least some activations to happen), and the result is passed through the activation algorithm producing one output for each hidden node.

Now that you have an idea about how Restricted Boltzmann Machine works, let's continue our Restricted Boltzmann Machine Tutorial and have a look at the steps involved in the training of RBM.

Training of Restricted Boltzmann Machine

The training of the Restricted Boltzmann Machine differs from the training of regular **neural networks** via stochastic gradient descent.

The Two main Training steps are:

- **Gibbs Sampling**

The first part of the training is called *Gibbs Sampling*. Given an input vector **v** we use **p(h|v)** for prediction of the hidden values **h**. Knowing the hidden values we use **p(v|h)** :

$$p(v_i = 1 | h) = \frac{1}{1 + e^{-(a_i + \sum_j h_j w_{ij})}} = \tilde{\sigma}(a_i + \sum_j h_j w_{ij})$$

for prediction of new input values **v**. This process is repeated *k* times. After *k* iterations, we obtain another input vector **v_k** which was recreated from original input values **v₀**.

$$p(h_i = 1 | v) = \frac{1}{1 + e^{-(b_i + \sum_j v_j w_{ji})}} = \sigma(b_i + \sum_j v_j w_{ji})$$

- **Contrastive Divergence step**

The update of the weight matrix happens during the *Contrastive Divergence* step.

Vectors $\mathbf{v}_0$ and $\mathbf{v}_k$ are used to calculate the activation probabilities for hidden values $\mathbf{h}_0$ and $\mathbf{h}_k$:

$$p(v_i = 1 | \mathbf{h}) = \frac{1}{1 + e^{-(a_i + \sum_j h_j w_{ij})}} = \tilde{O}(a_i + \sum_j h_j w_{ij})$$

The difference between the outer products of those probabilities with input vectors $\mathbf{v}_0$ and $\mathbf{v}_k$ results in the updated matrix :

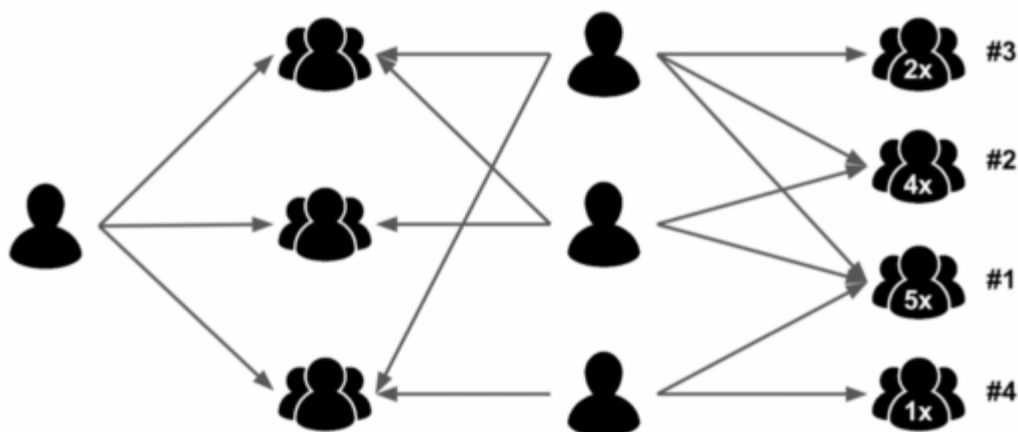
$$\Delta W = \mathbf{v}_0 \otimes P(\mathbf{h}_0 | \mathbf{v}_0) - \mathbf{v}_k \otimes P(\mathbf{h}_k | \mathbf{v}_k)$$

Using the update matrix the new weights can be calculated with gradient **ascent**, given by:

$$W_{new} = W_{old} + \Delta W$$

Now that you have an idea of what are Restricted Boltzmann Machines and the layers of RBM, let's move on with our Restricted Boltzmann Machine Tutorial and understand their working with the help of an example.

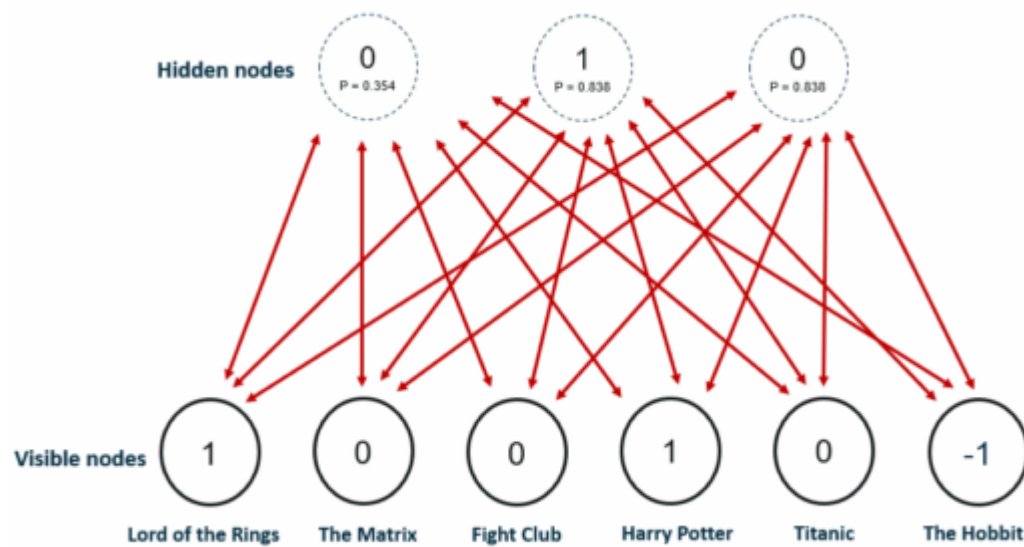
Restricted Boltzmann Machine Tutorial: Collaborative Filtering



RBMs have found applications in dimensionality reduction, classification, collaborative filtering and many more. They can be trained in either supervised or unsupervised ways, depending on the task.

Recognizing Latent factors in the Data

Let us assume that some people were asked to rate a set of movies in the scale of 1-5 and each movie could be explained in terms of a set of latent factors such as drama, fantasy, action and many more. Restricted Boltzmann Machines are used to analyze and find out these underlying factors.



The analysis of hidden factors is performed in a binary way, i.e, the user only tells if they liked (rating 1) a specific movie or not (rating 0) and it represents the inputs for the input/visible layer. Given the inputs, the RBM then tries to discover latent factors in the data that can explain the movie choices and each hidden neuron represents one of the latent factors.

Let us consider the following example where a user likes *Lord of the Rings* and *Harry Potter* but does not like *The Matrix*, *Fight Club* and *Titanic*. The Hobbit has not been seen yet so it gets a -1 rating. Given these inputs, the Boltzmann Machine may identify three hidden factors *Drama*, *Fantasy* and *Science Fiction* which correspond to the movie genres.

Using Latent Factors for Prediction

After the training phase, the goal is to predict a binary rating for the movies that had not been seen yet. Given the training data of a specific user, the network is able to identify the latent factors based on the user's preference and sample from Bernoulli distribution can be used to find out which of the visible neurons now become active.



The image shows the new ratings after using the hidden neuron values for the inference. The network identified *Fantasy* as the preferred movie genre and rated *The Hobbit* as a movie the user would like.

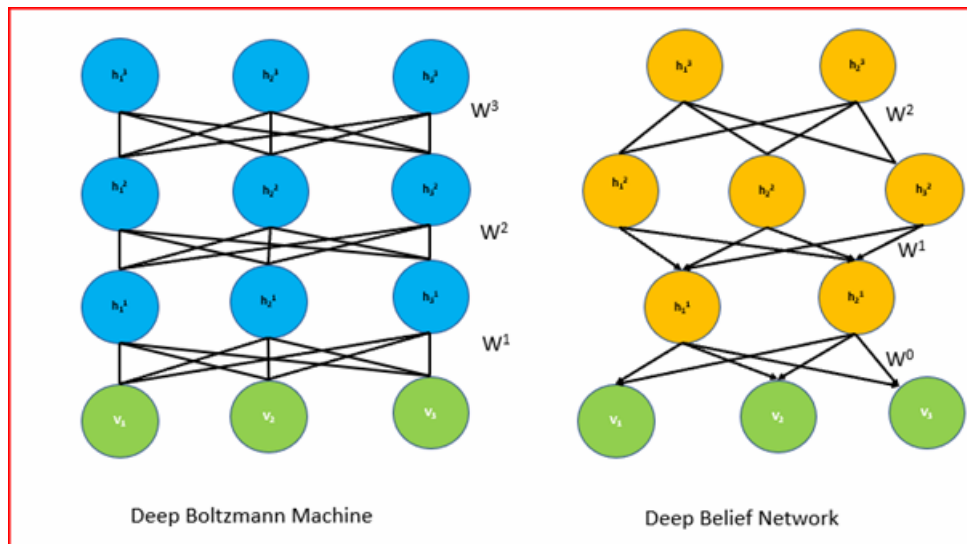
The process from **training** to the **prediction** phase goes as follows:

- Train the network on the data of all users
- During inference-time, take the training data of a specific user
- Use this data to obtain the activations of hidden neurons
- Use the hidden neuron values to get the activations of input neurons
- The new values of input neurons show the rating the user would give yet unseen movies

Deep Belief Network

The Architecture of DBN

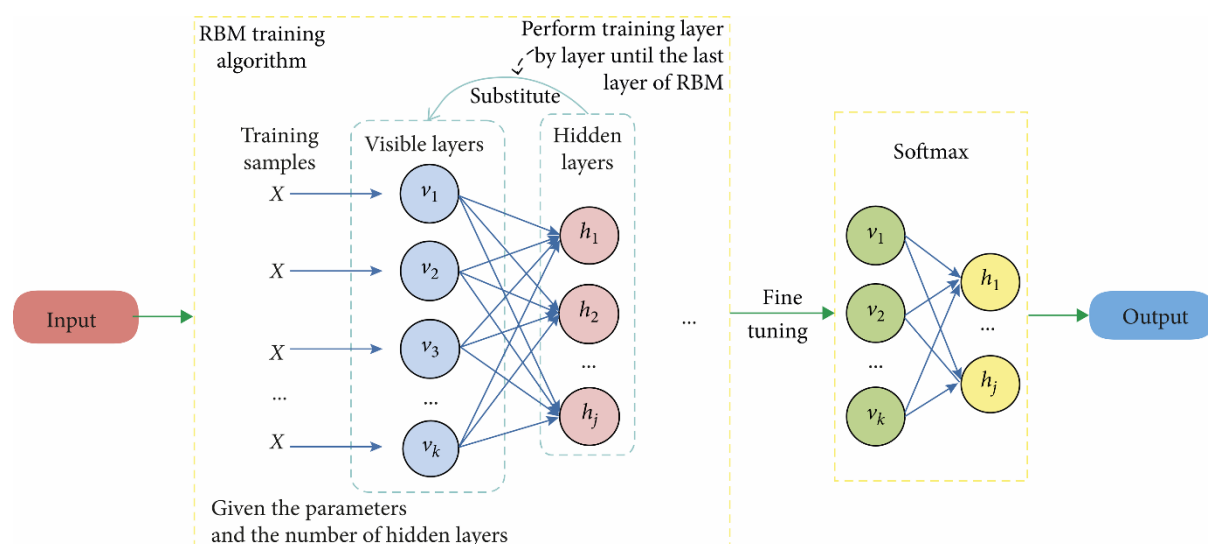
A series of constrained Boltzmann machines connected in a specific order make a **Deep Belief Network**. We supplement the result of the “output” layer of the Boltzmann machine as input to the next Boltzmann machine consecutively. Then we’ll train it until its convergence and apply the same until the completion of the whole network.



The undirected and symmetric connections between the top two levels of DBN form associative memory. The arrows pointing towards the layer closest to the data point to the relationships between all lower layers. Directed acyclic connections in the lower layers translate associative memory to observable variables. The lowest layer of visible units receives the input data. We can use Binary or actual data as input. Like *Restricted Boltzmann Machine (RBM)*, there are no intralayer connections. The hidden units represent features that encapsulate the data's correlations. A matrix of symmetrical weights W connects two layers. We'll link every unit in each layer to every other unit in the layer above it.

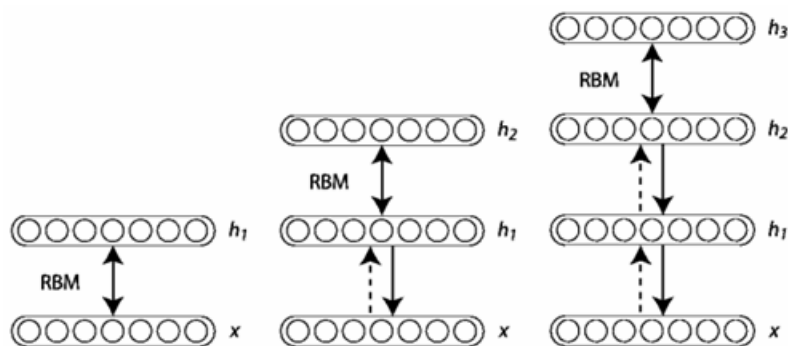
How does DBN work?

The first stage is to train a property layer that can directly gain input signals from pixels. In an alternate labeled data retired subcaste, learn the features of the preliminarily attained features by treating the values of this subcaste as pixels. The lower bound on the log-likelihood of the training data set improves every time a fresh subcaste of parcels or features that we add to the network.



Deep Belief Network's operational pipeline is as follows:

- We'll use the Greedy learning algorithm to pre-train DBN. For learning the top-down generative weights-the greedy learning method that employs a layer-by-layer approach. These generative weights determine the relationship between variables in one layer and variables in the layer above.
- On the top two hidden layers, we run numerous steps of Gibbs sampling in DBN. The top two hidden layers define the RBM thus, this stage is effectively extracting a sample from it.
- Then generate a sample from the visible units using a single pass of ancestral sampling through the rest of the model.
- We'll use a single bottom-up pass to infer the values of the latent variables in each layer. In the bottom layer, greedy pretraining begins with an observed data vector. It then oppositely [fine-tunes](#) the generative weights.



It's necessary to remember that constructing a Deep Belief Network necessitates training each RBM layer. Initially, we'll initiate the units and parameters for this purpose. In the Contrastive Divergence algorithm, there are two phases: positive and negative. We'll calculate the binary states of the hidden layers in the positive phase by computing the probabilities of weights and visible units. It is known as the positive phase since it enhances the likelihood of the training data set. The negative phase reduces the likelihood of the model producing samples. To train a complete Deep Belief Network, we'll employ the greedy learning technique. The greedy learning algorithm trains one RBM at a time until all of the RBMs are trained.

Unit 6

Dynamic Programming

Dynamic Programming (DP) in reinforcement learning is a collection of algorithms used to compute optimal policies by breaking down complex problems into simpler sub-problems and solving them step-by-step. DP is applicable when we have a full model of the environment, specifically the Markov Decision Process (MDP), with known states, actions, rewards, and transition probabilities.

Two main DP algorithms in reinforcement learning are **Policy Evaluation** and **Policy Iteration**.

1. **Policy Evaluation:** Calculates the value of each state (expected reward) given a policy, by iteratively updating the state values until they converge to accurate estimates.
2. **Policy Iteration:** Improves the policy by alternating between:
 - **Policy Evaluation:** Evaluates the current policy.
 - **Policy Improvement:** Uses the current value estimates to update the policy to one that performs better.

Example: Robot in a Grid World

Imagine a robot in a 3x3 grid world. The robot wants to reach a target cell (goal) in the bottom-right corner. It has to navigate through the grid with minimal steps, and it receives a reward of +1 when it reaches the goal and -1 for each move to encourage efficiency.

The robot can move **up, down, left, or right** (actions) with a slight chance of slipping into a neighboring cell.

Steps in Dynamic Programming:

1. **Initialize the Value Function:** Start with a random estimate of the values of all states.
2. **Policy Evaluation:** For each state, update the estimated value based on rewards and transition probabilities until the values stabilize (convergence).
3. **Policy Improvement:** Use the values of each state to decide the best action in each state. For example, the robot will favor actions that increase its chances of reaching the goal.
4. **Repeat:** Alternate between evaluation and improvement until the policy converges to an optimal one, where taking different actions cannot improve the expected reward.

Using DP, the robot systematically learns the best path to reach the goal efficiently, based on its state values and transitions.

Q Learning and Deep Q network

With the help of Robot in grid world example, elaborate Markov Decision Process.

A **Markov Decision Process (MDP)** is a mathematical framework used to model decision-making situations where outcomes are partly random and partly under the control of a decision-maker. MDPs are widely used in reinforcement learning.

An MDP consists of:

1. **States (S)**: All possible situations the system can be in.
2. **Actions (A)**: Choices available to the decision-maker at each state.
3. **Transition Probability (P)**: The probability of moving from one state to another after taking an action.
4. **Rewards (R)**: Immediate rewards received after transitioning from one state to another due to an action.
5. **Policy (π)**: A strategy or rule that tells the decision-maker which action to take in each state to maximize the expected reward over time.

Example: Robot in a Grid World

Imagine a robot navigating a grid world with 4x4 cells, where each cell represents a **state**. The robot can move **up, down, left, or right** (these are the **actions**).

- **Goal**: The robot aims to reach a specific cell (say, the bottom-right corner) to get a reward.
- **States (S)**: Each cell in the grid is a unique state.
- **Actions (A)**: Moving up, down, left, or right.
- **Transition Probability (P)**: The robot moves in the chosen direction with a high probability, but sometimes, it might end up in an adjacent cell due to slippage.
- **Reward (R)**: Reaching the target cell gives a positive reward; other cells might have small penalties to encourage efficient navigation.

By following an optimal **policy (π)**, the robot learns to reach the goal in the shortest time while maximizing its cumulative reward. This framework helps model scenarios where actions have probabilistic outcomes, making MDPs ideal for decision-making under uncertainty.

Simple reinforcement learning for Tic-Tac-Toe game

Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) is a revolutionary Artificial Intelligence methodology that combines [reinforcement learning](#) and [deep neural networks](#). By iteratively interacting with an environment and making choices that maximise cumulative rewards, it enables agents to learn sophisticated strategies. Agents are able to directly learn rules from sensory inputs thanks to DRL, which makes use of deep learning's ability to extract complex features from unstructured data. DRL relies heavily on [Q-learning](#), policy gradient methods, and [actor-critic systems](#). The notions of value networks, policy networks, and exploration-exploitation trade-offs are crucial. The uses for DRL are numerous and include [robotics](#), gaming, banking, and healthcare. Its development from Atari games to real-world difficulties emphasises how versatile and potent it is. Sample effectiveness, exploratory tactics, and safety considerations are difficulties. The collaboration aims to drive DRL responsibly, promising an inventive future that will change how decisions are made and problems are solved.

Core Components of Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) building blocks include all the aspects that power learning and empower agents to make wise judgements in their surroundings. Effective learning frameworks are produced by the cooperative interactions of these elements. The following are the essential elements:

- **Agent:** The decision-maker or learner who engages with the environment. The agent acts in accordance with its policy and gains experience over time to improve its ability to make decisions.
- **Environment:** The system outside of the agent that it communicates with. Based on the actions the agent does, it gives the agent feedback in the form of incentives or punishments.
- **State:** A depiction of the current circumstance or environmental state at a certain moment. The agent chooses its activities and makes decisions based on the state.
- **Action:** A choice the agent makes that causes a change in the state of the system. The policy of the agent guides the selection of actions. **Reward:** A scalar feedback signal from the environment that shows whether an agent's behaviour in a specific state is desirable. The agent is guided by rewards to learn positive behaviour.
- **Policy:** A plan that directs the agent's decision-making by mapping states to actions. Finding an ideal policy that maximises cumulative rewards is the objective.

- **Value Function:** This function calculates the anticipated cumulative reward an agent can obtain from a specific state while adhering to a specific policy. It is beneficial in assessing and contrasting states and policies.
- **Model:** A depiction of the dynamics of the environment that enables the agent to simulate potential results of actions and states. Models are useful for planning and forecasting.
- **Exploration-Exploitation Strategy:** A method of making decisions that strikes a balance between exploring new actions to learn more and exploiting well-known acts to reap immediate benefits (exploitation).
- **Learning Algorithm:** The process by which the agent modifies its value function or policy in response to experiences gained from interacting with the environment. Learning in DRL is fueled by a variety of algorithms, including Q-learning, policy gradient, and actor-critic.
- **Deep Neural Networks:** DRL can handle high-dimensional state and action spaces by acting as function approximators in deep neural networks. They pick up intricate input-to-output mappings.
- **Experience Replay:** A method that randomly selects from stored prior experiences (state, action, reward, and next state) during training. As a result, learning stability is improved and the association between subsequent events is decreased.

These core components collectively form the foundation of **Deep Reinforcement Learning**, empowering agents to learn strategies, make intelligent decisions, and adapt to dynamic environments.

How Deep Reinforcement Learning works?

In Deep Reinforcement Learning (DRL), an agent interacts with an environment to learn how to make optimal decisions. Steps:

1. Initialization: Construct an agent and set up the issue.
2. Interaction: The agent interacts with its surroundings through acting, which results in states and rewards.
3. Learning: The agent keeps track of its experiences and updates its method for making decisions.
4. Policy Update: Based on data, algorithms modify the agent's approach.
5. Exploration-Exploitation: The agent strikes a balance between using well-known actions and trying out new ones.
6. Reward Maximization: The agent learns to select activities that will yield the greatest possible total rewards.
7. Convergence: The agent's policy becomes better and stays the same over time.

8. Extrapolation: Skilled agents can use what they've learned in fresh circumstances.
9. Evaluation: Unknown surroundings are used to assess the agent's performance.
10. Use of the trained agent in practical situations.

Applications of Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) is used in a wide range of fields, demonstrating its adaptability and efficiency in solving difficult problems. Several well-known applications consist of:

1. Entertainment and gaming: DRL has mastered games like Go, Chess, and Dota 2 with ease. Also, it's used to develop intelligent, realistic game AI, which improves user experiences.
2. Robotics and autonomous systems: DRL allows robots to pick up skills like navigation, object identification, and manipulation. It is essential to the development of autonomous vehicles, drones, and industrial automation.
3. Finance and Trading: DRL enhances decision-making and profitability by optimising trading tactics, portfolio management, and risk assessment in financial markets.
4. Healthcare and Medicine: DRL helps develop individualised treatment plans, discover new medications, analyse medical images, identify diseases, and even perform robotically assisted procedures.
5. Energy Management: DRL makes sustainable energy solutions possible by optimising energy use, grid management, and the distribution of renewable resources.
6. Natural Language Processing (NLP): DRL enhances human-computer interactions by advancing dialogue systems, machine translation, text production, and sentiment analysis.
7. Recommendation Systems: By learning user preferences and adjusting to shifting trends, DRL improves suggestions in e-commerce, content streaming, and advertising.
8. Industrial Process Optimization: DRL streamlines supply chain management, quality control, and manufacturing procedures to cut costs and boost productivity.
9. Agricultural and Environmental Monitoring: Through enhancing crop production forecasting, pest control, and irrigation, DRL supports precision agriculture. Additionally, it strengthens conservation and environmental monitoring initiatives.
10. Education and Training: DRL is utilised to create adaptive learning platforms, virtual trainers, and intelligent tutoring systems that tailor learning experiences.